

Compte-rendu TP8

B - Contrôleur de disque

B1

- L'argument *block_size* est la taille en octets d'un bloc sur le disque.
- L'argument *latency* est le nombre de cycles d'attente pour effectuer un accès mémoire sur le disque.

B2

On a 128×128 pixels de 1 octet dans une image. Donc une image fait $2^7 \times 2^7 \approx 16\,000$ octets environ.

Une image occupe donc $(2^7 \times 2^7) / 2^9 = 2^5 = 32$ blocs.

B3

Les registres adressable du Block_Device sont:

- **BLOCK_DEVICE_BUFFER**: Une écriture sur ce registre permet de définir l'adresse du buffer utilisateur mais est ignorée si le Block_Device n'est pas en IDLE.
- **BLOCK_DEVICE_COUNT**: Une écriture sur ce registre permet de définir le nombre de blocs à traiter lors du transfert. Elle est ignorée si le Block_Device n'est pas en IDLE.
- **BLOCK_DEVICE_LBA** : Une écriture sur ce registre permet de définir l'adresse du premier bloc du transfert sur le disque. Elle est ignorée si le Block_Device n'est pas en IDLE.
- **BLOCK_DEVICE_OP**: Un accès en écriture sur ce registre lance un transfert du type que l'on y écrit. Si le Block_Device n'est pas en IDLE, l'écriture est ignorée.
- **BLOCK_DEVICE_STATUS**: Un accès en lecture sur ce registre provoque le reset de l'automate master à l'état Idle si le Block_Device est dans l'un des états suivants: READ_ERROR, READ_SUCCESS, WRITE_ERROR, WRITE_SUCCESS.
- **BLOCK_DEVICE_IRQ_ENABLE**: Si on écrit 1 dans ce registre, on active les IRQ. Si on écrit 0 on les désactive.
- **BLOCK_DEVICE_SIZE**: Une lecture de ce registre nous donne le nombre de blocs adressables sur le disque.
- **BLOCK_DEVICE_BLOCK_SIZE**: On peut lire la taille d'un bloc du disque sur ce registre.

B4

Les valeurs de l'état interne du contrôleur de disque qui peuvent être lues par le logiciel sont:

- **BLOCK_DEVICE_IDLE**: Attend un nouveau transfert.
- **BLOCK_DEVICE_BUSY**: Est occupé par un transfert.

- **BLOCK_DEVICE_READ_SUCCESS**: A réussi un transfert de lecture et attend l'acquittement de l'IRQ par le logiciel.
- **BLOCK_DEVICE_WRITE_SUCCESS**: A réussi un transfert d'écriture et attend l'acquittement de l'IRQ par le logiciel.
- **BLOCK_DEVICE_READ_ERROR**: Le transfert de lecture a échoué, et le contrôleur de disque attend que l'IRQ soit acquittée par le logiciel.
- **BLOCK_DEVICE_WRITE_ERROR**: Le transfert de lecture a échoué, et le contrôleur de disque attend que l'IRQ soit acquittée par le logiciel.

C - Architecture matérielle

On définit le timeout du composant PibusSegBcu à 1 000 000.

C1

L'utilisation du composant PibusBlockDevice impose de changer la valeur du timeout du composant PibusSegBcu car une transaction sur le bus avec le PibusBlockDevice prend beaucoup plus que 100 cycles. Un accès à la mémoire du disque est beaucoup plus lent qu'un transfert de donnée depuis la RAM. Le prix d'un accès sur le disque pouvant aller jusqu'à plusieurs millions de cycles pour un disque magnétique, on met le timeout à 10 000 000 de cycles.

C2

- Le segment IOC a pour adresse de base 0x92000000 et fait 32 octets.
- Le segment ICU a une longueur de $32 \times \text{nprocs}$ octets.
- Le segment TTY a une longueur de $16 \times \text{nprocs}$ octets.
- Le segment TIMER a une longueur de $16 \times \text{nprocs}$ octets.

C3

- Maîtres: DMA, IOC, $\text{nprocs} \times \text{PROC}$. On a donc 2 composants maîtres sans compter les processeurs.
- Cibles: DMA, IOC, Timer, TTY, ICU, FBF, RAM, ROM. On a donc 8 composants cibles.

C4

Le composant ICU reçoit comme lignes d'interruptions: 1 pour DMA, 1 pour IOC, 1 par processeur pour TIMER et 1 par processeur pour TTY.

Le composant ICU reçoit un total de $2 + 2 \times \text{nprocs}$ lignes d'interruptions entrantes.\

Le nombre de lignes d'interruptions sortantes est égal au nombre de coeurs.\

Les IRQs provenant des périphériques sont connectés au composant ICU à l'aide des signaux *signal_irq_dma*, *signal_irq_ioc*, *signal_irq_tim* et *signal_irq_tty_get*.

D - Code de boot

D1

Le pointeur de pile dépend du numéro de processeur car chaque processeur possède sa propre pile afin d'assurer la cohérence mémoire au sein d'un même programme.

D2

Afin de router les lignes d'interruptions du composant ICU vers les différents processeurs, on configure un mask pour chaque processeur permettant de filtrer les interruptions qui intéressent chaque processeur.

D3

- Proc 0: 0b 0000 0000 1111
- Proc 1: 0b 0000 0011 0000
- Proc 2: 0b 0000 1100 0000
- Proc 3: 0b 0011 0000 0000

E - Application logicielle de traitement d'image

E1

Les arguments de *ioc_read()* sont:

- Lba (logic block adress): Adresse du premier bloc à lire sur le disque.
- Buffer: Adresse destination dans l'espace utilisateur.
- Count: Nombre de blocs à lire.

Cet appel système lance un transfert de lecture sur le disque pour le copier dans le buffer utilisateur.

Il n'attend pas que le transfert soit terminé, il met simplement la variable *ioc_done* à 1 lorsqu'il est terminé.

Cet appel système est bloquant lorsque le composant IOC n'est pas libre (*ioc_lock*=1).

E2

L'appel système *ioc_completed()* ne prend pas d'arguments et attend que la variable *ioc_done* soit égale à 0.

Ensuite il met les variables *ioc_lock* et *ioc_done* à 0.

E3 Lors de la simulation, si on met l'option -SNOOP 0, seule la première image reste affichée malgré que l'on passe à l'image suivante.

Ceci est dû au cache de données qui pense que l'image est à jour alors qu'elle a été modifiée par le composant IOC. La première itération, le cache ne fait pas de HIT donc lit la mémoire en RAM, mais les suivantes le cache fait HIT et il n'y a plus de lecture en RAM pour mettre à jour *buf_in*. Le résultat affiché ne change donc pas.\

L'attribut volatile ne règle pas le problème car il n'a pas d'effet sur la décision du cache mais sur le stockage d'un variable dans un registre du processeur. Même si l'on force l'accès mémoire avec l'attribut volatile, on fera toujours un HIT sur le cache ce qui empêchera de récupérer la bonne valeur.

E4

Les conditions qui font sortir l'automate SNOOP_FSM de l'état IDLE sont la détection de 3 types d'écritures externes:

- L'écriture externe a matché avec une ligne de cache local.
- L'écriture externe a matché avec une ligne de cache demandée.
- L'écriture externe a matché avec un envoie d'adresse avec lisc.

La stratégie mise en oeuvre en cas de hit externe est une invalidation de la ligne de cache concernée.

E5

La détection de plusieurs hit externes consécutifs pose problème car l'invalidation en cours n'a pas été finalisée.

Cela provoque le passage du Snoop_fsm en mode panique et va invalider tout le DCACHE.

E6

	Chargement	Seuillage	Affichage
Image 1	41 813	553 913	112 247
Image 2	40 651	554 129	112 083
Image 3	40 705	554 129	112 083
Image 4	40 705	554 129	112 083

F - Exécution sur architecture multi-processeurs

F1

La seule phase qui va être parallélisée est la phase de filtrage qui est découpée entre les 4 processeurs.

La phase d'affichage et de lecture dans le disque ne sont pas paralléliser car elle ne permettent qu'un transfert en même temps.

F2

Le mécanisme général permettant de séquentialiser les 4 transferts demandés par les 4 processeurs est le suivant:

Chaque processeur fait sa propre demande de lecture dans le disque, l'écrit dans son buffer de sortie puis fait sa propre demande d'écriture sur le FrameBuffer.

Le filtrage de l'image est donc indépendant des autres processeurs, mais les étapes de lecture et d'affichage de l'image forceront les processeurs à s'attendre entre eux.

F3

`_ioc_get_lock` permet d'attendre que la variable `_ioc_lock` passe à 0 puis la remet à 1 pour que l'appelant de l'appel système puisse utiliser l'IOC.

Elle permet donc de prendre le verrou sur l'IOC.

F4

La fonction système qui libère ce verrou est la fonction `_ioc_completed` qui doit être appelé par l'utilisateur ayant fait un appel à la fonction `_ioc_read` ou `_ioc_write` afin de libérer le verrou du composant IOC

```
(_ioc_lock = 0).\
```

G - Réalisation matérielle du LL/SC

G1

On réalise l'enregistrement d'une adresse réservée par l'instruction LL plutôt du côté processeur que dans la mémoire car cela permet d'avoir pour chaque processeur 1 registre de réservation d'adresse qui peut être vérifié par le mécanisme de Snoop.

G2

Dans un scénario où deux processeurs P0 et P1 réservent une même adresse X et que le processeur P1 exécute une instruction SC en premier, le processeur P0 sera informé de l'invalidation de sa réservation lorsqu'il exécutera SC qui renverra la valeur 0 et sa ligne de cache correspondant à l'adresse X sera invalidée.

Le processeur devra recommencer la réservation jusqu'à ce que l'appel à SC renvoie 1.

G3

Si on effectue une simulation en enlevant le mécanisme de SNOOP (-SNOOP 0), il manque la deuxième bande de l'image et elle reste figée malgré que l'on entre un caractère.

Ceci s'explique par le fait que les lignes de caches correspondant au buffer *buf_in* n'ont pas été invalidées par le mécanisme de Snoop, le processeur considère donc le buffer comme à jour alors qu'il a été modifié par le composant IOC.

G4

Dans ce TP, on a mis en évidence l'utilisation du mécanisme de Snoop dans le partage d'un même composant par plusieurs processeurs à l'aide d'un verrou ainsi que la cohérence mémoire entre les processeurs qui doit être respectée.

Le mécanisme de Snoop permet donc d'assurer une cohérence mémoire qui est nécessaire au partage des ressources dans un système multi-processeurs.